

DIFFERENCE-AWARE RETRIEVAL POLICIES FOR IMITATION LEARNING

Anonymous authors
Anonymous submission

ABSTRACT

Behavior cloning is a simple and widely used route to imitation learning, but purely parametric policies remain exposed to the closed-loop states created by compounding prediction error. This paper demonstrates that the demonstration corpus can be retained as an inference-time resource to reduce this vulnerability while preserving the data and supervision assumptions of standard behavior cloning. We introduce Difference-Aware Retrieval Policies for Imitation Learning (DARP), a semi-parametric policy class that reparameterizes imitation around local neighborhood geometry rather than a single global state-to-action map. For each query state, DARP retrieves k nearest expert states, conditions prediction on the retrieved states, their expert actions, and query-relative difference vectors, and aggregates the resulting neighbor-conditioned action predictions. The method requires no additional interaction, online expert correction, task-specific supervision, or supervision beyond the demonstrations used for ordinary behavior cloning. Across continuous-control and robotic-manipulation domains, and across both low-dimensional state inputs and high-dimensional visual-feature representations, DARP improves over standard behavior cloning by 15-46%. These results establish difference-aware retrieval and permutation-invariant local aggregation as a practical mechanism for strengthening imitation policies while retaining the deployment, data, and supervision profile of behavior cloning. Code and demos will be released upon acceptance.

1 INTRODUCTION

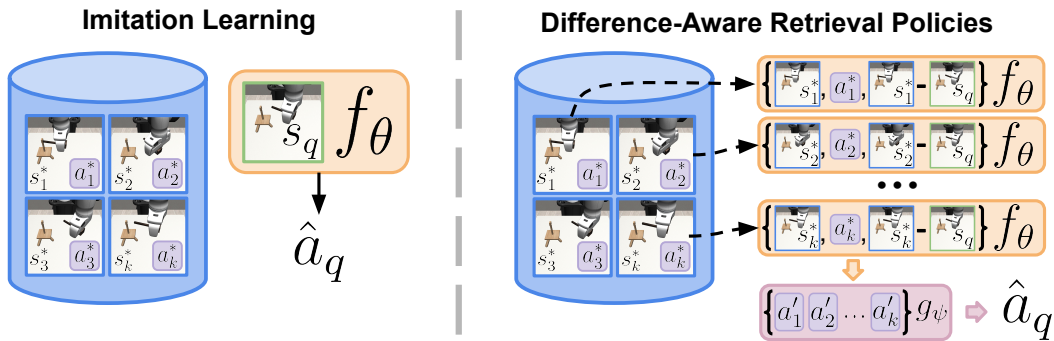


Figure 1: **Overview of DARP:** Standard BC (left) consumes the expert dataset during training and then deploys a purely parametric state-to-action map. DARP (right) preserves the expert state-action corpus as an inference-time resource, retrieves a local neighborhood for each query state s_q , and predicts actions from difference-aware tuples that jointly encode the neighbor state, the associated expert action, and the displacement from the query.

Behavior cloning (BC) (Pomerleau, 1991) remains one of the most direct and consequential mechanisms for acquiring complex robotic skills from expert demonstrations (Zhao et al., 2023; Chi et al., 2024; Black et al., 2024; Chung et al., 2014). Its appeal lies in a deliberately restrictive supervision model: the learner observes demonstrated state-action pairs and receives no further interaction. The

same restriction also creates the canonical closed-loop failure mode of BC, particularly in long-horizon tasks (Ross et al., 2011). Small prediction errors compound, displace the agent from the demonstrated state distribution, and induce covariate shift (Spencer et al., 2021; Ross et al., 2011). Once the rollout enters low-density or out-of-distribution regions, a purely parametric BC policy is only weakly constrained by its training objective and can become high variance, unstable, and ultimately task-failing.

A substantial literature mitigates compounding error (Ross et al., 2011; Venkatraman et al., 2015; Ke et al., 2024b; Levine et al., 2020), yet many effective remedies relax the standard BC regime by introducing simulators, interactive expert correction, suboptimal experience, or task-specific structure. This paper takes a complementary position: the original demonstration dataset already contains operational information that ordinary parametric BC discards after training. The central question is therefore sharper than whether additional supervision can improve robustness; it is whether the expert state-action corpus already available for supervised imitation can be reused to reduce policy variance at deployment.

The statistical limitation of BC is precise: the supervised objective constrains predictions at expert states but leaves policy variation between, near, and beyond observed samples largely unregulated. A learned policy can therefore achieve low training error while exhibiting undesirable local oscillation around the demonstration manifold. A principled remedy is to impose *smoothness*, requiring nearby states to induce compatible actions (Kobayashi, 2022; Asadi et al., 2018; Ke et al., 2024a; Chen et al., 2024). Smoothness directly targets the variation that destabilizes rollouts, but existing strategies, discussed in Section 4, introduce distinct tradeoffs: data augmentation does not by itself guarantee local consistency, global continuity priors can blur legitimately distinct behaviors, temporal penalties act along trajectories rather than across state-space neighborhoods, and explicit graph penalties add smoothness hyperparameters that must be tuned.

A second perspective contrasts global parametric learning with local retrieval-based prediction. Global supervised policies (Black et al., 2024; Zhao et al., 2023; Chi et al., 2024) compress the demonstration corpus into a single learned function, an efficient representation that can become brittle when rollout states depart from the training distribution. Local methods (Pari et al., 2022; Mansimov & Cho, 2018; Salzberg & Aha, 1994) instead condition prediction on neighborhoods of similar examples and thereby preserve empirical structure in expert behavior at inference time. Locality anchors decisions in observed data, but standard local prediction is not sufficient for imitation: performance depends on the retrieval metric, naive averaging can collapse distinct action modes, and representations based only on absolute neighbor states omit the query-relative transformation that links a retrieved example to the current state.

We introduce **Difference-Aware Retrieval Policies for Imitation Learning (DARP)**, a semi-parametric architecture that combines the empirical anchoring of retrieval with the expressive capacity of learned prediction. At inference time, DARP (Fig. 1) retrieves k nearest expert states for the current query, constructs tuples consisting of each neighbor state, its expert action, and the query-to-neighbor difference vector, and predicts neighbor-conditioned candidate actions. A permutation-invariant aggregation operator then produces the executed action. This design keeps each decision tied to the demonstrated corpus while enabling a learned model to interpret relative local geometry. It also induces local consistency without additional data, online supervision, task-specific knowledge, or an explicitly tuned smoothness penalty. In spectral terms, the aggregation step approximates a Laplacian filter on the k -NN (k -nearest neighbor) graph of expert states, producing parameter-free smoothing determined by the density and geometry of the demonstrations.

The resulting contribution is both architectural and empirical. Analytically, we formalize the relationship between neighborhood aggregation and Laplacian regularization, showing that the architecture implicitly applies a fixed low-pass spectral filter that suppresses high-frequency policy variation. Empirically, we show that this mechanism delivers consistent improvements in the evaluated settings: across MuJoCo continuous-control tasks, Robosuite and RoboCasa manipulation tasks, and image-feature-based imitation, DARP improves over standard BC by 15–46%. The evaluation further demonstrates compatibility with high-dimensional representations and richer policy classes, including transformers and Gaussian mixture models, while ablations identify the difference-vector representation and permutation-invariant aggregation as essential components. Together, these results establish difference-aware retrieval as a broadly applicable design principle for strengthening imitation policies under standard demonstration-only supervision.

2 DIFFERENCE-AWARE RETRIEVAL POLICIES FOR IMITATION LEARNING

This section develops DARP as a semi-parametric imitation-learning architecture that combines global neural prediction with local retrieval over the expert corpus. The construction preserves the data and supervision requirements of standard behavior cloning while introducing an architectural mechanism for variance reduction and local rollout stability. We first make the underlying regularization principle explicit, then show that the same smoothing effect can be obtained implicitly through neighborhood aggregation. Readers primarily interested in implementation may proceed to Section 2.5 for a concise algorithmic summary before the empirical evaluation in Section 3.

The development proceeds in three steps. Section 2.1 formalizes the behavior-cloning setting, and Section 2.2 introduces a neighborhood-manifold regularized objective that exposes the desired smoothness structure. Section 2.3 then proves that an analogous Laplacian-smoothing effect can be produced by the policy *architecture* rather than by an additional loss term. Finally, Section 2.4 instantiates this principle as DARP, a retrieval-augmented architecture compatible with modern neural predictors and action-distribution models.

2.1 PRELIMINARIES: BEHAVIOR CLONING FOR IMITATION LEARNING

We work in the standard finite-horizon imitation-learning setting, represented as a Markov Decision Process (MDP) $\mathcal{M} = \{\mathcal{S}, \mathcal{A}, P_0\}$ with state space $\mathcal{S}$, action space $\mathcal{A}$, and initial-state distribution P_0 . A *policy* maps states to action distributions, $f_\theta : \mathcal{S} \rightarrow \Delta_{\mathcal{A}}$; when the function class is discussed abstractly, we omit the parameter subscript. The learner receives expert demonstrations $\mathcal{D}^* = \{(s_j^*, a_j^*)\}$, where s^* and a^* denote states and actions from the expert dataset. Behavior cloning (Pomerleau, 1991) casts imitation as supervised conditional-density estimation and learns f_θ by optimizing $\arg \max_\theta \mathbb{E}_{(s^*, a^*) \sim \mathcal{D}^*} [\log(f_\theta(a^* | s^*))]$. Although f_θ can be instantiated by expressive generative models (Lipman et al., 2023; Chi et al., 2024), we use a Gaussian parameterization initially to keep the regularization argument explicit¹.

2.2 WARM-UP: NEIGHBOR MANIFOLD REGULARIZED IMITATION LEARNING

Behavior cloning minimizes supervised imitation loss on expert states sampled from $\mathcal{D}^*$, but that objective provides no direct constraint away from the demonstrated manifold. During rollout, accumulated errors can therefore move the agent into regions where an overparameterized BC policy is weakly determined by the data and may produce arbitrary actions (Ross & Bagnell, 2010).

The missing constraint is the relationship among neighboring demonstrations. Standard BC supervises pointwise predictions but does not require nearby expert states, or their associated actions, to induce locally coherent policy outputs. We therefore introduce a neighborhood-regularized objective that explicitly enforces *local consistency*: nearby expert states should receive compatible predictions. The resulting loss $\mathcal{L}_{\text{MRIL}}$ augments the standard imitation objective $\mathcal{L}_{\text{BC}}$ with a smoothness penalty $\mathcal{L}_S$, requiring the learned policy to respect the empirical geometry of the demonstration dataset rather than relying only on isolated pointwise supervision.

$$\mathcal{L}_{\text{MRIL}}(f) = \underbrace{\mathbb{E}_{(s^*, a^*) \sim \mathcal{D}^*} [\ell(f(s^*), a^*)]}_{\text{supervised risk } (\mathcal{L}_{\text{BC}})} + \lambda \underbrace{\mathbb{E}_{s^* \sim \mathcal{D}^*} \left[\sum_{i \in \mathcal{N}_k(s^*)} w_i(s^*) \|f(s^*) - f(s_i^*)\|_2^2 \right]}_{\text{smoothness regularizer } (\mathcal{L}_S)}, \quad (1)$$

Here $\ell(f(s^*), a^*)$ denotes the supervised imitation loss, $\mathcal{N}_k(s^*)$ is the set of k nearest expert neighbors of s^* , and $w_i(s^*)$ are normalized kernel weights computed from state differences, $w_i(s^*) \propto K_\Delta\left(\frac{\|s_i^* - s^*\|}{h}\right)$. As summarized below and proved in Appendix A.1.1, this term is a manifold-regularization penalty: it implements Laplacian smoothing on the neighborhood graph and suppresses high-frequency policy variation over expert states. Under the theorem assumptions, the regularized objective improves estimator variance, policy smoothness, and closed-loop stability relative to vanilla BC.

¹We show that this can be relaxed in Section 2.4

Theorem 1 (Manifold Regularized BC ($\mathcal{L}_{\text{MRIL}}$) improves over vanilla BC ($\mathcal{L}_{\text{BC}}$)). Let $f^* : \mathcal{S} \rightarrow \mathcal{A}$ be the true, underlying expert policy, assumed to be C^2 -smooth on a compact state space $\mathcal{S}$. Let $f : \mathcal{S} \rightarrow \mathcal{A}$ denote the learned policy estimator. Consider two estimators trained on expert demonstrations:

1. **Vanilla BC:** a global supervised model minimizing

$$\mathcal{L}_{\text{BC}}(f) = \mathbb{E}_{(s^*, a^*) \sim \mathcal{D}^*} [\ell(f(s^*), a^*)].$$

2. **MRIL:** a neighbor-based estimator minimizing

$$\mathcal{L}_{\text{MRIL}}(f) = \mathcal{L}_{\text{BC}}(f) + \lambda \mathbb{E}_{s^* \sim \mathcal{D}^*} \left[\sum_{i \in \mathcal{N}_k(s^*)} w_i(s^*) \|f(s^*) - f(s_i^*)\|_2^2 \right],$$

where $w_i(s^*)$ are the kernel weights defined above and $\lambda > 0$.

Then, under the smoothness assumption on f , the following hold:

- (i) Variance reduction: The Laplacian penalty in MRIL acts as a data-dependent Tikhonov regularizer, yielding smaller estimator variance than vanilla BC.
- (ii) Smoothness guarantee: Minimizers of $\mathcal{L}_{\text{MRIL}}$ satisfy a uniform bound on the local Lipschitz constant of f , whereas vanilla BC admits interpolants with arbitrarily large Lipschitz constants between training states.
- (iii) Policy stability: In a closed-loop rollout, the deviation recursion

$$\Delta_{t+1} \leq L_s \Delta_t + L_a \|f(s_t) - f^*(s_t^*)\|^2$$

accumulates error linearly for vanilla BC, but sublinearly for MRIL, since the smoothness regularizer enforces $\|f(s^*) - f(s^{j*})\| = O(\|s^* - s^{j*}\|)$ for neighbors s^*, s^{j*} .

Thus, under the theorem’s smoothness assumptions, MRIL provides stronger variance, smoothness, and stability guarantees than vanilla BC.

Proof sketch. Appendix A.1.1 provides the full proof. The argument has three components: the smoothness term is first expressed as a graph-Laplacian penalty on the k -nearest neighbor (k -NN) affinity graph induced by the kernel weights w_i ; in the large-sample limit, this graph penalty converges to the weighted Dirichlet energy (Belkin & Niyogi, 2008; Zhou et al., 2003); and minimizing the Dirichlet energy yields almost-everywhere local Lipschitz control while also acting as Tikhonov regularization, thereby supporting rollout stability and reducing estimator variance with controlled bias. $\square$

The regularizer therefore does more than penalize isolated pairwise disagreements; it aligns the learned policy with the geometry of the data manifold. In high-density regions, it reduces the local Lipschitz constant of f , so small state perturbations induce correspondingly small and coherent action changes. This mechanism explains how local smoothness can improve reliability on in-distribution (ID) states and provide structured behavior for nearby out-of-distribution (OOD) states.

2.3 IMPLICIT MANIFOLD REGULARIZATION VIA IN-CONTEXT ARCHITECTURES

Although MRIL supplies the desired smoothness, it introduces two practical costs. The weight λ must be tuned to balance supervised accuracy against smoothness, and the modified objective changes the optimization problem relative to ordinary BC. These costs motivate the central architectural question: *can the benefits of MRIL (Eq. 1) be obtained by modifying the policy architecture rather than the training objective?*

We answer this question with a retrieval-based architecture that induces *implicit* manifold regularization (iMRIL) while retaining the standard imitation objective. As illustrated in Fig. 2, iMRIL realizes the low-pass effect of Laplacian smoothing without introducing λ or adding an explicit regularization term. Section 2.4 then converts this implicit construction into the practical DARP algorithm.

² L_s and L_a are the Lipschitz constants of the environment transition dynamics with respect to state and action, respectively.

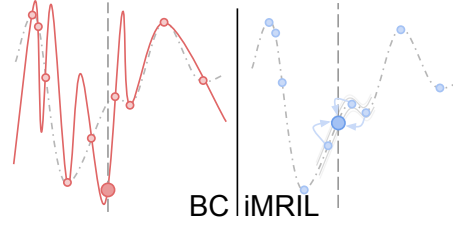


Figure 2: **iMRIL induces Laplacian smoothing through architecture.** Neighborhood aggregation dampens high-frequency variation and enforces local consistency, whereas standard BC can admit jagged interpolants because it has no explicit or implicit smoothness constraint.

iMRIL architecture: The key operation in iMRIL is to transfer neighborhood aggregation from the objective into the policy parameterization. Instead of training a standard feed-forward predictor $f(s)$ with the neighborhood-regularized objective in Eq. 1, the action predictor $\hat{f}$ is defined as an aggregation over retrieved training states, while the loss remains the standard imitation loss. Specifically, iMRIL learns a per-state predictor f_θ such that the aggregated predictor $\hat{f}(s^*) = \frac{1}{k} \sum_{i \in \mathcal{N}_k(s^*)} f_\theta(s_i^*)$, evaluated over nearest-neighbor states $\{s_i^*\}_{i \in \mathcal{N}_k(s^*)}$, predicts the corresponding expert action a^* . Training therefore optimizes:

$$\arg \min_{\theta} \mathbb{E}_{(s^*, a^*) \sim \mathcal{D}^*} \left[\left\| \underbrace{\left(\frac{1}{k} \sum_{i \in \mathcal{N}_k(s^*)} f_\theta(s_i^*) \right)}_{\hat{f}(s^*)} - a^* \right\|_2 \right] \quad (2)$$

At deployment, a query state s_q is processed by retrieving its k nearest training states and applying the same aggregation rule, $\hat{a} = \hat{f}(s_q) = \frac{1}{k} \sum_{i \in \mathcal{N}_k(s_q)} f_\theta(s_i^*)$. Section 2.4 shows that the representation supplied to f_θ is empirically decisive, which motivates the difference-aware parameterization used by DARP.

This architecture supervises the post-aggregation predictor rather than each unaggregated neighbor prediction in isolation. Consequently, the aggregation operator suppresses arbitrary non-smooth local predictions and yields the same qualitative benefits as the explicit regularizer in Section 2.2. The next result makes this connection precise by relating iMRIL to Laplacian regularization.

Equivalence between iMRIL and MRIL: We state the equivalence result here and give the complete proof in Appendix Section A.1.

Theorem 2 (iMRIL is parameter-free Laplacian regularization for BC (MRIL)). *Consider the symmetric normalized k -NN graph Laplacian L (defined in Section 2.2), with eigenpairs $\{(\mu_j, u_j)\}_{j=1}^n$, where $0 = \mu_1 \leq \mu_2 \leq \dots \leq \mu_n \leq 2$.*

The minimizers of the explicit MRIL objective (Section 2.2) and the implicit iMRIL objective (Section 2.3) have the following closed form expansions

$$f_{\text{MRIL}} = \sum_{j=1}^n \frac{1}{1 + \lambda \mu_j} \langle a^*, u_j \rangle u_j \quad \hat{f}_{\text{iMRIL}} = \sum_{j=1}^n (1 - \mu_j) \langle f, u_j \rangle u_j$$

iMRIL's neighbor aggregation step applies the fixed spectral filter $\phi_{\text{iMRIL}}(\mu) = 1 - \mu$ to the graph Laplacian L , preserving low-frequency modes and suppressing high-frequency modes. The congruence between $\hat{f}_{\text{iMRIL}}$ and f_{MRIL} shows that iMRIL is equivalent to a built-in form of Laplacian smoothing (MRIL) with effective $\lambda \approx 1$ in normalized units. Unlike explicit regularization, this implicit filter requires no additional hyperparameter tuning.

Proof sketch. We defer the full derivation to Appendix Section A.1.2. The explicit regularizer admits a spectral solution obtained by diagonalizing the k -NN Laplacian, which yields a filter of the form $(1 + \lambda \mu)^{-1}$ on each eigenmode. The implicit objective can be expressed as neighbor aggregation $\hat{f} = S f$ with $S = D^{-1} A$, the random-walk matrix, which shares the same eigenvectors and applies the fixed filter $1 - \mu$. Both operations act as graph low-pass filters: modes with small eigenvalues, corresponding to smooth variation across the data manifold, are largely preserved, whereas modes with large eigenvalues, corresponding to rapid high-variance fluctuation between neighbors, are strongly damped. Thus iMRIL performs Laplacian smoothing implicitly, reducing variance and enforcing local consistency without tuning λ . $\square$

The implicit-smoothing interpretation complements, rather than replaces, policy learning. The learned predictor f_θ models how expert actions vary under local perturbations and produces neighbor-specific candidate actions; aggregation then controls variance by smoothing those candidates over the retrieved neighborhood. In this decomposition, learned local adaptation supplies accuracy, while architectural averaging supplies stability.

2.4 DIFFERENCE-AWARE RETRIEVAL POLICIES: A PRACTICAL INSTANTIATION OF IMRIL FOR IMITATION LEARNING

We now instantiate iMRIL as a practical imitation-learning algorithm. The implementation retains the objective in Eq. 2 and specifies the two architectural choices that determine empirical performance: the representation consumed by the neighbor predictor and the permutation-invariant mechanism used to aggregate neighbor-conditioned predictions.

2.4.1 DIFFERENCE-BASED PARAMETERIZATION OF f_θ

Eq. 2 leaves the representation of f_θ unspecified. DARP resolves this ambiguity by making local displacement an explicit input. Because retrieval should capture how an expert action changes under a query-relative perturbation, the predictor receives the neighbor state, the neighbor action, and the vector from the query to that neighbor. Thus, in **Difference-Aware Retrieval Policies (DARP)**, each candidate action is parameterized as $a'_i = f_\theta(s_i^*, a_i^*, \Delta s_i = s_i^* - s_q)$ for a query s_q and retrieved neighbor (s_i^*, a_i^*) . This representation enables the model to transform an observed expert action according to local state displacement rather than predicting solely from an absolute state.

Let $\mathcal{N}_k(s_q)$ denote the indices of the k nearest neighbors under a distance function $d(s_q, s_i^*)$.³ Given these retrieved tuples, DARP predicts the action for query state s_q by applying the aggregation operation introduced in Section 2.3:

$$\hat{a}_q = f_{\text{DARP}}(s_q) = \frac{1}{k} \sum_{i \in \mathcal{N}_k(s_q)} a'_i \quad (3)$$

$$= \frac{1}{k} \sum_{i \in \mathcal{N}_k(s_q)} f_\theta(s_i^*, a_i^*, \Delta s_i = s_i^* - s_q). \quad (4)$$

Training uses the expert dataset $\mathcal{D}^*$ directly and minimizes the supervised discrepancy between the aggregated prediction and the demonstrated action:

$$\arg \min_{\theta} \mathbb{E}_{(s_q^*, a_q^*) \sim \mathcal{D}^*} [\|f_{\text{DARP}}(s_q^*) - a_q^*\|^2] \quad (5)$$

The optimization updates the predictor parameters θ so that the aggregated action $\hat{a}_q$ matches the expert action a_q^* . The objective remains deliberately simple and permits standard feedforward or convolutional parameterizations of f_θ . Section 3 shows that the difference-vector representation is not an incidental implementation choice but a central contributor to performance. At inference time, DARP executes actions by retrieving k nearest neighbors and applying the aggregation rule in Eq. 3.

2.4.2 GOING BEYOND LINEAR AGGREGATION

Averaging neighbor predictions, $\hat{a}_q = \frac{1}{k} \sum_{i \in \mathcal{N}_k(s_q)} a'_i$, is one member of a broader family of permutation-invariant aggregation functions $g_\psi(\{a'_i\}_{i \in \mathcal{N}_k(s_q)})$. The aggregator can be implemented by set-compliant neural architectures such as Set Transformer (Lee et al., 2019) or DeepSets (Zaheer et al., 2017), yielding the generalized prediction rule $\hat{a}_q = g_\psi(\{f_\theta(s_i^*, a_i^*, \Delta s_i = s_i^* - s_q)\}_{i \in \mathcal{N}_k(s_q)})$. This generalization increases expressivity and supports richer action-distribution models than the Gaussian assumption implicit in the L_2 objective of Eq. 5. Instead of predicting a single action $\hat{a}_q$, DARP can predict distribution parameters α for $p(a_q; \alpha)$, including the means, covariances, and weights of a Gaussian mixture model (Pignat & Calinon, 2019) or the score function of a diffusion model (Chi et al., 2024). The resulting policy can be trained by maximum likelihood for multimodal action distributions:

³In our work we use the Euclidean distance in a pre-trained embedding space, although other neighborhood functions are also applicable. We refer the reader to Appendix Section A.2.1 for a thorough discussion of design decisions in constructing neighborhood sets via retrieval.

$$\arg \max_{\theta} \mathbb{E}_{(s_q^*, a_q^*) \sim \mathcal{D}^*} [\log p(a_q^*; \alpha_{\theta}(s_q^*))], \text{ where } \alpha_{\theta}(s_q^*) = g_{\psi} \left(\{f_{\theta}(s_i^*, a_i^*, \Delta s_i = s_i^* - s_q^*)\}_{i \in \mathcal{N}_k(s_q^*)} \right) \quad (6)$$

At inference, the policy retrieves neighbors for s_q , constructs $\alpha_{\theta}(s_q) = g_{\psi} \left(\{f_{\theta}(s_i^*, a_i^*, \Delta s_i = s_i^* - s_q)\}_{i \in \mathcal{N}_k(s_q)} \right)$, and samples $\hat{a}_q \sim p(\cdot; \alpha_{\theta}(s_q))$. Appendix Section A.3 provides training pseudocode.

2.5 SUMMARY

This section establishes the technical basis for DARP. Explicit local consistency in BC corresponds to manifold regularization, or Laplacian smoothing, and yields variance, smoothness, and stability improvements under the conditions of Theorem 1. Rather than adding this regularizer and tuning λ , DARP incorporates neighborhood aggregation into the architecture, inducing the same low-pass smoothing effect implicitly (Theorem 2). Operationally, for each expert pair (s_q^*, a_q^*) , DARP retrieves k nearest neighbors from $\mathcal{D}^*$, computes $\Delta s_i = s_i^* - s_q^*$, maps each tuple $(s_i^*, a_i^*, \Delta s_i)$ through f_{θ} to obtain candidate actions, and aggregates those candidates with a permutation-invariant function g_{ψ} . The resulting policy is trained with the standard imitation-learning objective.

3 EXPERIMENTAL EVALUATION

We evaluate DARP through three questions that connect the proposed architecture to its empirical claims: **Q1**: Does DARP consistently improve over standard behavior cloning?, **Q2**: Does DARP remain effective with more complex state representations and action distributions?, and **Q3**: Which architectural components account for the observed gains? The evaluation spans low-dimensional state inputs, high-dimensional image features, and richer action representations across MuJoCo continuous-control tasks, Robosuite and RoboCasa robotic-manipulation tasks, and additional discontinuous environments that stress-test retrieval-based prediction.

3.1 BASELINE COMPARISONS AND TASK DESCRIPTIONS

MuJoCo Tasks: The MuJoCo (Todorov et al., 2012; Fu et al., 2020) benchmarks evaluate continuous-control locomotion across several embodiments on a flat plane: Hopper (single-legged hopping robot), Walker (bipedal humanoid), Ant (quadruped), and HalfCheetah (biped).

Robosuite Tasks: The Robosuite (Zhu et al., 2020) benchmarks evaluate single-arm object manipulation. Stack requires placing a smaller cube on a larger cube; Threading requires inserting a thin needle-like tool into a small ring; and Square Peg requires placing a square block with a central hole onto a square peg.

RoboCasa Tasks: The RoboCasa (Nasiriany et al., 2024) benchmarks evaluate single-arm manipulation in randomized kitchen scenes. Drawer requires closing an open drawer, Door requires closing an open microwave door, and Stove requires twisting a knob to turn off a burner.

Baseline Comparisons: We compare DARP with baselines that isolate retrieval, local regression, parametric supervision, in-context conditioning, and explicit smoothness: (1) **R&P (Sridhar et al., 2025)**: executes the action associated with the nearest neighbor; (2) **LWR (Pari et al., 2022)**: applies locally weighted regression over retrieved neighbors; (3) **BC**: trains a standard parametric behavior-cloning policy; (4) **REGENT (Sridhar et al., 2025)**: conditions a transformer-based in-context policy on retrieved neighbors; and (5) **MRIL**: implements the explicitly smoothed objective described in Section 2.2.

3.2 CAN DARP CONSISTENTLY OUTPERFORM STANDARD BEHAVIOR CLONING? (Q1)

We first test the central hypothesis in settings with low-dimensional state representations, where neighbor distances are well defined and interpretable. The experiments include MuJoCo locomotion and Robosuite/RoboCasa manipulation, with manipulation demonstrations generated using MimicGen

(Mandlekar et al., 2023). Unless otherwise specified, the aggregation function g is the simple average of all neighbor-conditioned action predictions a' .

Method	Hopper	Ant	Walker	HalfCheetah
R&P (Sridhar et al., 2025)	711.82 $\pm$ 85.63	-305.97 $\pm$ 76.42	419.18 $\pm$ 50.21	-178.64 $\pm$ 29.75
LWR (Pari et al., 2022)	1703.78 $\pm$ 245.95	846.59 $\pm$ 216.06	1484.91 $\pm$ 356.54	1945.82 $\pm$ 567.26
BC	2313.65 $\pm$ 203.75	2376.20 $\pm$ 339.43	2658.40 $\pm$ 274.08	1063.23 $\pm$ 371.08
REGENT (Sridhar et al., 2025)	1819.39 $\pm$ 186.24	-302.10 $\pm$ 146.67	507.01 $\pm$ 76.10	169.85 $\pm$ 63.10
MRIL	2793.63 $\pm$ 156.41	3869.08 $\pm$ 241.00	4370.96 $\pm$ 168.13	701.58 $\pm$ 195.08
DARP	3545.57 $\pm$ 3.54	4383.28 $\pm$ 266.37	4894.01 $\pm$ 75.12	5515.41 $\pm$ 841.33
DARP Set Transformer	2965.86 $\pm$ 103.08	4063.79 $\pm$ 218.80	4752.42 $\pm$ 109.23	3417.85 $\pm$ 764.57

Table 1: **DARP achieves the strongest MuJoCo scores across all four low-dimensional locomotion tasks.** Scores compare DARP, DARP Set Transformer, BC, retrieval baselines, REGENT, and explicit MRIL on MuJoCo environments. Values are averages over 100 independent trials with 95% confidence intervals. Parametric policies use Multi-Layer Perceptrons; diffusion-policy results are reported in Section 8.

The low-dimensional results support the main empirical claim: DARP improves over standard BC across all evaluated environments. In robotic manipulation, the gains range from 15-25 percentage points, and in locomotion DARP produces substantial score increases relative to BC (Table 1 and Table 2). The comparison also clarifies the role of the architecture. Purely non-parametric retrieval methods (R&P and LWR) are not sufficient on these tasks; MRIL usually improves over vanilla BC, but the strongest scores in this suite are obtained by the implicit, difference-aware DARP architecture.

Because Section 2.4 introduces a representation-level modification rather than a domain-specific assumption, we next test whether the same retrieval principle extends to higher-dimensional inputs such as visual features.

3.3 CAN DARP HANDLE MORE COMPLEX STATE REPRESENTATION AND ACTION DISTRIBUTIONS? (Q2)



Figure 3: The “DrawerClose” RoboCasa task, where the robotic arm must close an open drawer.



Figure 4: The “Threading” Robosuite task, where the robotic arm must insert a needle-like implement into a small hole.

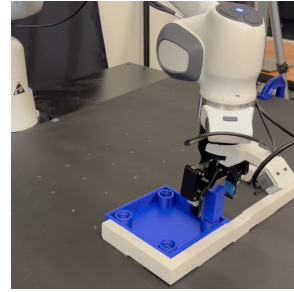


Figure 5: Real-world FurnitureBench square-table assembly task, where the robot must pick up a table leg and screw it into a corner hole in the tabletop.

Method	RoboCasa			Robosuite			Real
	Drawer	Door	Stove	Stack	Thrd.	Peg	Sq. Table
BC	54	29	28	47	37	46	44
DARP	85	45	43	72	63	62	92

Table 2: Low-dimensional robotic-manipulation results comparing DARP with BC across RoboCasa, Robosuite, and a real-world FurnitureBench task. Scores are success percentages (RoboCasa & Robosuite: 100 trials; Real: 50 trials). DARP outperforms BC on every task and more than doubles BC on the real-world square-table task. RoboCasa and Robosuite policies use Multi-Layer Perceptrons; the real-world result uses diffusion policy, as discussed in Section 8.

High-Dimensional Visual Input Representations. To test whether DARP applies beyond compact state vectors, we evaluate simulated manipulation policies using R3M image embeddings (Nair et al., 2022). This setting preserves the same imitation-learning protocol while replacing low-dimensional states with high-dimensional visual feature vectors (Table 3). DARP outperforms standard BC on all three Robosuite vision tasks, and its average improvement, $\sim 35\%$, exceeds the corresponding average improvement on low-dimensional Robosuite state inputs ($\sim 22\%$). The result indicates that difference-aware retrieval remains effective when neighborhood structure is computed in a learned visual-feature space.

Method	Stack	Thrd.	Peg
BC	44	38	17
DARP	75	76	52

Table 3: Success rates (%) for BC and DARP on vision-based Robosuite tasks using high-dimensional visual features.

Multi-modal Action Distributions. DARP also extends to tasks with multimodal action structure: on Push-T, it improves over behavior cloning by more than 20%. Full details are provided in Appendix A.2.2.

3.4 HOW DO DIFFERENT ARCHITECTURAL COMPONENTS CONTRIBUTE TO DARP’S PERFORMANCE GAINS? (Q3)

Ablation Study: We isolate the architectural sources of DARP’s gains through a systematic ablation study. The variants compare (1) standard DARP; (2) DARP without neighbor actions; (3) an ensemble of 10 BC agents; (4) DARP with randomly selected neighbors instead of distance-based retrieval; (5) DARP using the L_2 norm of the distance vector; (6) the BC baseline, which conditions only on the query state s_q ; (7) DARP using the query state rather than the query-neighbor difference vector; and (8) DARP with a permutation-dependent, rather than permutation-invariant, aggregator over the a' predictions. Figure 6 reports the results.

The ablation results identify the difference-vector representation and permutation-invariant aggregation as the principal drivers of DARP’s performance. Neighbor actions contribute more mod-

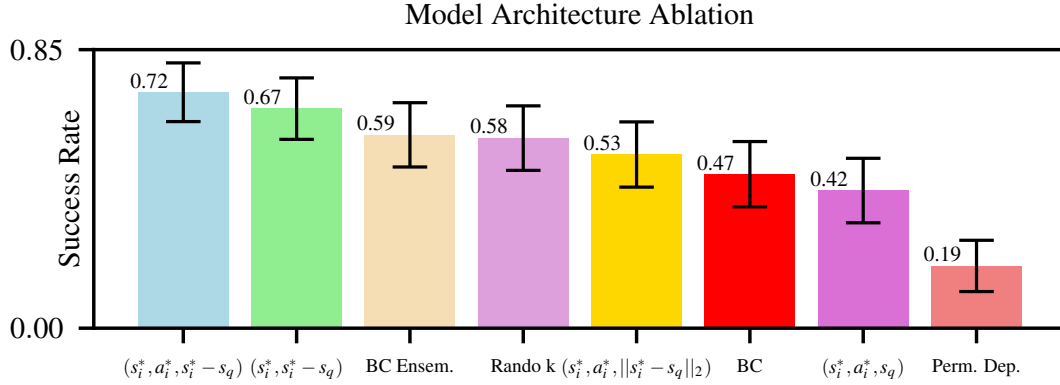


Figure 6: **Distance vectors and permutation invariance are central to DARP’s performance.** Architectural ablations on Robosuite Stack show that the full tuple $(s_i^*, a_i^*, s_i^* - s_q)$ and permutation-invariant aggregation provide the largest contributions to success. Success rates are averaged across 100 trials with 95% confidence intervals.

estly, whereas random neighbor selection performs poorly, confirming that retrieval must be guided by a meaningful state-distance metric. Replacing the permutation-invariant aggregator g with a permutation-dependent alternative substantially degrades performance, supporting the set-structured design of the architecture.

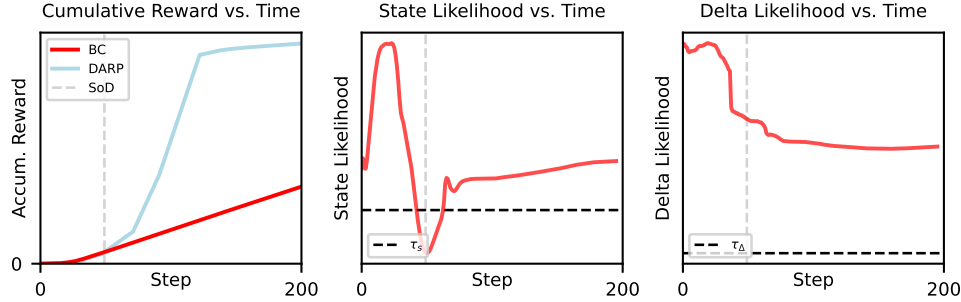


Figure 7: **Cumulative rewards for BC and DARP on the Robosuite stack task illustrate initially identical rollouts that diverge as BC fails the task and DARP succeeds.** A vertical dashed line indicates the step in which the two diverge, labeled “SoD”. At the SoD, the state likelihood is $< \tau_s$ (OOD), but the delta likelihood is $> \tau_\Delta$ (in distribution).

Divergence Analysis: To better understand DARP’s success over standard BC, we analyze the point of divergence in rollouts in which the latter fails but the former succeeds. We identify the “step of divergence” as the point at which DARP and BC begin to receive a significantly different reward. We define τ_s and τ_Δ as the 1st percentile of likelihoods of the training set (that is, 1% of the deltas seen at training time are less likely than τ_Δ). In all six different rollouts across two different tasks (the Robosuite Stack task and the MuJoCo Hopper task), the query state at the SoD has a state likelihood of $< \tau_s$ but a delta likelihood of $> \tau_\Delta$. This result bolsters our hypothesis that DARP gains occur partly due to improved prediction on slightly out-of-distribution states due to reparameterization in terms of difference vectors to neighbors. (see Figure 7 for plots of reward drift, SoDs, and state and delta likelihood for one task.) See Appendix A.2.4 for additional experiments regarding DARP robustness.

4 RELATED WORK

Non-Parametric Imitation Learning Methods: Non-parametric and retrieval-based imitation methods have shown that local structure in demonstration data can be highly informative for policy learning. VINN Pari et al. (2022) studies locally weighted regression for imitation and demonstrates strong performance in image-embedding spaces, while MiDiGaP von Hartz et al. (2025) uses mixtures of Gaussian processes to model multimodal trajectories and support rapid generalization. SEABO Lyu et al. (2024) uses retrieval in offline RL by assigning rewards to transitions that are close to retrieved

neighbors. FlowRetrieval Lin et al. (2024), STRAP Memmel et al. (2025), and Behavior Retrieval Du et al. (2023) combine non-parametric retrieval with finetuning from large unlabeled datasets, enabling generalization through test-time training. DARP differs from these approaches by representing each retrieved example as a difference-aware tuple $(s_i^*, a_i^*, s_i^* - s_q)$ and by learning a semi-parametric policy rather than relying solely on non-parametric aggregation or test-time adaptation. This design retains the variance-reduction benefits of local methods while preserving the expressivity and generalization capacity of parametric policies.

Smoothness-Constrained Policy Learning: A growing body of work imposes explicit smoothness constraints to improve policy stability and robustness. L2C2 Kobayashi (2022) studies model-free RL under local Lipschitz-continuity constraints, improving smoothness and noise robustness without sacrificing expressiveness, while Asadi et al. (2018) propose a related Lipschitz-constrained methodology for model-based RL. CCIL Ke et al. (2024a) extends these ideas to imitation learning by generating synthetic corrective labels with a Lipschitz-constrained dynamics model, and related smoothness constraints have been scaled to humanoid controllers Chen et al. (2024) to reduce deployment shakiness. DARP pursues the same stability objective through a different mechanism: smoothness is induced implicitly by the retrieval architecture and standard imitation objective rather than by an additional explicit constraint or corrective-label procedure.

In-Context Learning Methods: Recent work also studies retrieval through the lens of in-context imitation learning. REGENT Sridhar et al. (2025) investigates retrieval-augmented generalization by conditioning a causal transformer on retrieved states, actions, and rewards, while DPT Lee et al. (2023) uses supervised transformer pretraining to predict actions from query states and in-context datasets, thereby learning exploratory behavior. Other in-context architectures include ICRT Fu et al. (2024), Instant Policy Vosylus & Johns (2025), and KAT Di Palo & Johns (2024). These methods primarily target rapid adaptation to new tasks and environments. In contrast, DARP focuses on improving performance and stability within standard imitation learning by reusing the original demonstration corpus through difference-aware retrieval.

5 CONCLUSION

We introduced **D**ifference-**A**ware **R**etrieval **P**olicies for Imitation Learning (**DARP**) (DARP), a nearest-neighbor-based algorithm that reparameterizes the imitation learning problem in terms of relative differences between query states and their nearest neighbors, rather than learning direct state-to-action mappings. We prove that our method implicitly achieves Laplacian smoothing. Our experimental evaluation across diverse domains, including continuous control and robotic manipulation, validates three key hypotheses. First, DARP consistently outperforms standard behavior cloning when using low-dimensional state representation. Second, DARP maintains performance across different state representations, action distribution modeling requirements, and task complexities, with improvements ranging from 15-46% across tested scenarios. Third, architectural ablations reveal that distance vectors and permutation-invariant aggregation are crucial components to our algorithm.

6 REPRODUCIBILITY STATEMENT

Supplementary source code, policy and environment configuration files, MuJoCo experiment data, scripts for generating expert demonstrations for Robosuite tasks via MimicGen, and modality transformation code will be released upon acceptance. Results will be identical to those in the paper on NVIDIA L40 and L40s GPUs, with the exception of results that require the use of a transformer (REGENT, Set Transformer), which are non-deterministic and may differ slightly from reported numbers.

REFERENCES

- Kavosh Asadi, Dipendra Misra, and Michael Littman. Lipschitz continuity in model-based reinforcement learning. In *International Conference on Machine Learning*. PMLR, PMLR, 2018.
- Mikhail Belkin and Partha Niyogi. Towards a theoretical foundation for laplacian-based manifold methods. *Journal of Computer and System Sciences*, 74(8):1289–1308, 2008.

-
- Kevin Black, Noah Brown, Danny Driess, Adnan Esmail, Michael Equi, Chelsea Finn, Niccolo Fusai, Lachy Groom, Karol Hausman, Brian Ichter, Szymon Jakubczak, Tim Jones, Liyiming Ke, Sergey Levine, Adrian Li-Bell, Mohith Mothukuri, Suraj Nair, Karl Pertsch, Lucy Xiaoyang Shi, James Tanner, Quan Vuong, Anna Walling, Haohuan Wang, and Ury Zhilinsky. π_0 : A vision-language-action flow model for general robot control, 2024. URL <https://arxiv.org/abs/2410.24164>.
- Zixuan Chen, Xialin He, Yen-Jen Wang, Qiayuan Liao, Yanjie Ze, Zhongyu Li, S. Shankar Sastry, Jiajun Wu, Koushil Sreenath, Saurabh Gupta, and Xue Bin Peng. Learning smooth humanoid locomotion through lipschitz-constrained policies. *arxiv preprint arXiv:2410.11825*, 2024.
- Cheng Chi, Zhenjia Xu, Siyuan Feng, Eric Cousineau, Yilun Du, Benjamin Burchfiel, Russ Tedrake, and Shuran Song. Diffusion policy: Visuomotor policy learning via action diffusion. *The International Journal of Robotics Research*, 2024.
- Fan R. K. Chung. *Spectral Graph Theory*, volume 92 of *CBMS Regional Conference Series in Mathematics*. American Mathematical Society, 1997. doi: 10.1090/cbms/092.
- Michael Jae-Yoon Chung, Maxwell Forbes, Maya Cakmak, and Rajesh PN Rao. Accelerating imitation learning through crowdsourcing. In *2014 IEEE International Conference on Robotics and Automation (ICRA)*, pp. 4777–4784. IEEE, 2014.
- Norman Di Palo and Edward Johns. Keypoint action tokens enable in-context imitation learning in robotics. In *Proceedings of Robotics: Science and Systems (RSS)*, 2024.
- Maximilian Du, Suraj Nair, Dorsa Sadigh, and Chelsea Finn. Behavior retrieval: Few-shot imitation learning by querying unlabeled datasets. In *Proceedings of Robotics: Science and Systems (RSS)*, 2023.
- Justin Fu, Aviral Kumar, Ofir Nachum, George Tucker, and Sergey Levine. D4rl: Datasets for deep data-driven reinforcement learning, 2020.
- Letian Fu, Huang Huang, Gaurav Datta, Lawrence Yunliang Chen, William Chung-Ho Panitch, Fangchen Liu, Hui Li, and Ken Goldberg. In-context imitation learning via next-token prediction. *arXiv preprint arXiv:2408.15980*, 2024.
- Liyiming Ke, Yunchu Zhang, Abhay Deshpande, Siddhartha Srinivasa, and Abhishek Gupta. Ccil: Continuity-based data augmentation for corrective imitation learning. In *The Twelfth International Conference on Learning Representations*, 2024a. URL <https://openreview.net/forum?id=LQ6LQ8f4y8>.
- Liyiming Ke, Yunchu Zhang, Abhay Deshpande, Siddhartha S. Srinivasa, and Abhishek Gupta. CCIL: continuity-based data augmentation for corrective imitation learning. In *The Twelfth International Conference on Learning Representations, ICLR 2024, Vienna, Austria, May 7-11, 2024*. OpenReview.net, 2024b. URL <https://openreview.net/forum?id=LQ6LQ8f4y8>.
- Taisuke Kobayashi. L2c2: Locally lipschitz continuous constraint towards stable and smooth reinforcement learning. In *2022 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pp. 4032–4039. IEEE, IEEE, October 2022.
- Jonathan Lee et al. Supervised pretraining can learn in-context reinforcement learning. In *Advances in Neural Information Processing Systems*, volume 36, pp. 43057–43083, 2023.
- Juho Lee, Yoonho Lee, Jungtaek Kim, Adam Kosior, Seungjin Choi, and Yee Whye Teh. Set transformer: A framework for attention-based permutation-invariant neural networks. In *Proceedings of the 36th International Conference on Machine Learning*, pp. 3744–3753, 2019.
- Sergey Levine, Aviral Kumar, George Tucker, and Justin Fu. Offline reinforcement learning: Tutorial, review, and perspectives on open problems, 2020. URL <https://arxiv.org/abs/2005.01643>.

Li-Heng Lin, Yuchen Cui, Amber Xie, Tianyu Hua, and Dorsa Sadigh. Flowretrieval: Flow-guided data retrieval for few-shot imitation learning. In Pulkrit Agrawal, Oliver Kroemer, and Wolfram Burgard (eds.), *Conference on Robot Learning, 6-9 November 2024, Munich, Germany*, volume 270 of *Proceedings of Machine Learning Research*, pp. 4084–4099. PMLR, 2024. URL <https://proceedings.mlr.press/v270/lin25a.html>.

Yaron Lipman, Ricky T. Q. Chen, Heli Ben-Hamu, Maximilian Nickel, and Matthew Le. Flow matching for generative modeling. In *International Conference on Learning Representations (ICLR) 2023*, 2023. URL <https://arxiv.org/abs/2210.02747>. Originally appeared as arXiv preprint arXiv:2210.02747.

Jiafei Lyu, Xiaoteng Ma, Le Wan, Runze Liu, Xiu Li, , and Zongqing Lu. Seabo: A simple search-based method for offline imitation learning. In *International Conference on Learning Representations*, 2024.

Ajay Mandlekar, Soroush Nasiriany, Bowen Wen, Iretiayo Akinola, Yashraj Narang, Linxi Fan, Yuke Zhu, and Dieter Fox. Mimicgen: A data generation system for scalable robot learning using human demonstrations. In *7th Annual Conference on Robot Learning*, 2023.

Elman Mansimov and Kyunghyun Cho. Simple nearest neighbor policy method for continuous control tasks. In *International Conference on Learning Representations (ICLR) 2018*, 2018. Under review as a conference paper at ICLR 2018.

Marius Memmel, Jacob Berg, Bingqing Chen, Abhishek Gupta, and Jonathan Francis. Strap: Robot sub-trajectory retrieval for augmented policy learning. In *The Thirteenth International Conference on Learning Representations*, 2025.

Suraj Nair, Aravind Rajeswaran, Vikash Kumar, Chelsea Finn, and Abhinav Gupta. R3M: A universal visual representation for robot manipulation. In Karen Liu, Dana Kulic, and Jeffrey Ichnowski (eds.), *Conference on Robot Learning, CoRL 2022, 14-18 December 2022, Auckland, New Zealand*, volume 205 of *Proceedings of Machine Learning Research*, pp. 892–909. PMLR, 2022. URL <https://proceedings.mlr.press/v205/nair23a.html>.

Soroush Nasiriany, Abhiram Maddukuri, Lance Zhang, Adeet Parikh, Aaron Lo, Abhishek Joshi, Ajay Mandlekar, and Yuke Zhu. Robocasa: Large-scale simulation of everyday tasks for generalist robots. In *Robotics: Science and Systems (RSS)*, 2024.

Jyothish Pari, Nur Muhammad Shafuallah, Sridhar Pandian Arunachalam, and Lerrel Pinto. The surprising effectiveness of representation learning for visual imitation. In *Robotics: Science and Systems (RSS)*. Robotics: Science and Systems Foundation, June 2022. doi: 10.15607/RSS.2022.XVIII-052.

Emmanuel Pignat and Sylvain Calinon. Bayesian gaussian mixture model for robotic policy imitation. *IEEE Robotics and Automation Letters*, 2019. URL <https://arxiv.org/pdf/1904.10716.pdf>. Preprint version.

Dean Pomerleau. Efficient training of artificial neural networks for autonomous navigation. *Neural Comput.*, 3(1):88–97, 1991. doi: 10.1162/NECO.1991.3.1.88. URL <https://doi.org/10.1162/neco.1991.3.1.88>.

Stéphane Ross and Drew Bagnell. Efficient reductions for imitation learning. In *Proceedings of the thirteenth international conference on artificial intelligence and statistics*, pp. 661–668. JMLR Workshop and Conference Proceedings, 2010.

Stéphane Ross, Geoffrey J. Gordon, and Drew Bagnell. A reduction of imitation learning and structured prediction to no-regret online learning. In Geoffrey J. Gordon, David B. Dunson, and Miroslav Dudík (eds.), *Proceedings of the Fourteenth International Conference on Artificial Intelligence and Statistics, AISTATS 2011, Fort Lauderdale, USA, April 11-13, 2011*, volume 15 of *JMLR Proceedings*, pp. 627–635. JMLR.org, 2011. URL <http://proceedings.mlr.press/v15/ross11a/ross11a.pdf>.

-
- Steven L. Salzberg and David W. Aha. Learning to catch: Applying nearest neighbor algorithms to dynamic control tasks. In P. Cheeseman and R. W. Oldford (eds.), *Selecting Models from Data: Artificial Intelligence and Statistics IV*, volume 89 of *Lecture Notes in Statistics*, pp. 321–328. Springer, 1994. doi: 10.1007/978-1-4612-2660-4_33. URL https://link.springer.com/chapter/10.1007/978-1-4612-2660-4_33.
- Jonathan C. Spencer, Sanjiban Choudhury, Arun Venkatraman, Brian D. Ziebart, and J. Andrew Bagnell. Feedback in imitation learning: The three regimes of covariate shift. *CoRR*, abs/2102.02872, 2021. URL <https://arxiv.org/abs/2102.02872>.
- Kaustubh Sridhar, Souradeep Dutta, Dinesh Jayaraman, and Insup Lee. REGENT: A retrieval-augmented generalist agent that can act in-context in new environments. In *The Thirteenth International Conference on Learning Representations (ICLR)*, 2025. Oral presentation.
- Emanuel Todorov, Tom Erez, and Yuval Tassa. Mujoco: A physics engine for model-based control. In *2012 IEEE/RSJ international conference on intelligent robots and systems*, pp. 5026–5033. IEEE, 2012.
- Arun Venkatraman, Martial Hebert, and J. Andrew Bagnell. Improving multi-step prediction of learned time series models. In Blai Bonet and Sven Koenig (eds.), *Proceedings of the Twenty-Ninth AAAI Conference on Artificial Intelligence, January 25-30, 2015, Austin, Texas, USA*, pp. 3024–3030. AAAI Press, 2015. doi: 10.1609/AAAI.V29I1.9590. URL <https://doi.org/10.1609/aaai.v29i1.9590>.
- Jan Ole von Hartz, Adrian Röfer, Joschka Boedecker, and Abhinav Valada. The unreasonable effectiveness of discrete-time gaussian process mixtures for robot policy learning. *arXiv preprint arXiv:2505.03296*, 2025.
- Vitalis Vosylius and Edward Johns. Instant policy: In-context imitation learning via graph diffusion. In *Proceedings of the International Conference on Learning Representations (ICLR)*, 2025.
- Manzil Zaheer, Satwik Kottur, Siamak Ravanbakhsh, Barnabas Poczos, Ruslan R Salakhutdinov, and Alexander J Smola. Deep sets. In I. Guyon, U. V. Luxburg, S. Bengio, H. Wallach, R. Fergus, S. Vishwanathan, and R. Garnett (eds.), *Advances in Neural Information Processing Systems 30*, pp. 3391–3401. Curran Associates, Inc., 2017. URL <http://papers.nips.cc/paper/6931-deep-sets.pdf>.
- Tony Z. Zhao, Vikash Kumar, Sergey Levine, and Chelsea Finn. Learning fine-grained bimanual manipulation with low-cost hardware. In *Proceedings of Robotics: Science and Systems (RSS)*, 2023.
- Dengyong Zhou, Olivier Bousquet, Thomas Navin Lal, Jason Weston, and Bernhard Schölkopf. Learning with local and global consistency. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 16, 2003.
- Yuke Zhu, Josiah Wong, Ajay Mandlekar, Roberto Martín-Martín, Abhishek Joshi, Soroush Nasiriany, Yifeng Zhu, and Kevin Lin. robosuite: A modular simulation framework and benchmark for robot learning. In *arXiv preprint arXiv:2009.12293*, 2020.

A APPENDIX

A.1 LEMMAS AND PROOFS

A.1.1 PROOF OF THEOREM 1

To prove Theorem 1, we first start with a well-known result (Chung, 1997; Zhou et al., 2003; Belkin & Niyogi, 2008).

Lemma 1 (Smoothness regularizer as k -NN graph Laplacian penalty). *Let $\{s_1^*, \dots, s_n^*\}$ be the expert states with corresponding predicted actions $f(s_i^*) \in \mathbb{R}^{d_a}$. For each i , let $\mathcal{N}_k(s_i^*)$ denote the indices of the k -nearest neighbors of s_i^* (excluding i). Define asymmetric weights*

$$\tilde{W}_{ij} = \begin{cases} w_j(s_i^*), & \text{if } j \in \mathcal{N}_k(s_i^*), \\ 0, & \text{otherwise,} \end{cases}$$

and construct a symmetric affinity matrix

$$W_{ij} = \frac{1}{2} (\tilde{W}_{ij} + \tilde{W}_{ji}).$$

Let D be the degree matrix with $D_{ii} = \sum_j W_{ij}$, and define the k -NN graph Laplacian $L = D - W$.

Then the smoothness regularizer can be written as the quadratic form

$$\mathcal{L}_S(f) = \frac{1}{n} \sum_{i=1}^n \sum_{j \in \mathcal{N}_k(s_i^*)} w_j(s_i^*) \|f(s_i^*) - f(s_j^*)\|^2 \propto \text{Tr}(F^\top L F),$$

where $F = [f(s_1^*), f(s_2^*), \dots, f(s_n^*)]^\top \in \mathbb{R}^{n \times d_a}$. Equivalently, in the scalar case,

$$\mathcal{L}_S(f) \propto f^\top L f.$$

Corollary 1 (Continuum limit of smoothness regularizer). *Assume states $\{s_i^*\}_{i=1}^n$ are sampled i.i.d. from a smooth density $p(s^*)$ supported on an m -dimensional C^2 manifold $\mathcal{M} \subset \mathbb{R}^d$. Let W be the symmetrized k -NN affinity matrix constructed from a kernel K_Δ with bandwidth h , and let $L = D - W$ be the graph Laplacian.*

If $n \rightarrow \infty$, $h \rightarrow 0$, and $nh^{m+2} \rightarrow \infty$, then the normalized quadratic form converges to the weighted Dirichlet energy:

$$\frac{1}{n^2 h^{m+2}} \text{Tr}(F^\top L F) \longrightarrow C_K \int_{\mathcal{M}} \|\nabla_{\mathcal{M}} f(s)\|_2^2 p(s)^2 d\text{vol}(s),$$

where $C_K > 0$ is a constant depending only on the kernel K_Δ .

Theorem 1 (Manifold Regularized BC ($\mathcal{L}_{\text{MRIL}}$) improves over vanilla BC ($\mathcal{L}_{\text{BC}}$)). *Let $f^* : \mathcal{S} \rightarrow \mathcal{A}$ be the true, underlying expert policy, assumed to be C^2 -smooth on a compact state space $\mathcal{S}$. Let $f : \mathcal{S} \rightarrow \mathcal{A}$ denote the learned policy estimator. Consider two estimators trained on expert demonstrations:*

1. **Vanilla BC:** a global supervised model minimizing

$$\mathcal{L}_{\text{BC}}(f) = \mathbb{E}_{(s^*, a^*) \sim \mathcal{D}^*} [\ell(f(s^*), a^*)].$$

2. **MRIL:** a neighbor-based estimator minimizing

$$\mathcal{L}_{\text{MRIL}}(f) = \mathcal{L}_{\text{BC}}(f) + \lambda \mathbb{E}_{s^* \sim \mathcal{D}^*} \left[\sum_{i \in \mathcal{N}_k(s^*)} w_i(s^*) \|f(s^*) - f(s_i^*)\|_2^2 \right],$$

where $w_i(s^*)$ are the kernel weights defined above and $\lambda > 0$.

Then, under the smoothness assumption on f , the following hold:

- (i) Variance reduction: The Laplacian penalty in MRIL acts as a data-dependent Tikhonov regularizer, yielding smaller estimator variance than vanilla BC.

(ii) Smoothness guarantee: Minimizers of $\mathcal{L}_{\text{MRIL}}$ satisfy a uniform bound on the local Lipschitz constant of f , whereas vanilla BC admits interpolants with arbitrarily large Lipschitz constants between training states.

(iii) Policy stability: In a closed-loop rollout, the deviation recursion

$$\Delta_{t+1} \leq L_s \Delta_t + L_a \|f(s_t) - f^*(s_t^*)\|^4$$

accumulates error linearly for vanilla BC, but sublinearly for MRIL, since the smoothness regularizer enforces $\|f(s^*) - f(s'^*)\| = O(\|s^* - s'^*\|)$ for neighbors s^*, s'^* .

Thus, under the theorem's smoothness assumptions, MRIL provides stronger variance, smoothness, and stability guarantees than vanilla BC.

Proof. We prove each claim in turn.

(i) Variance reduction. The Laplacian penalty in $\mathcal{L}_{\text{MRIL}}$ is

$$\sum_{i,j} W_{ij} \|f(s_i^*) - f(s_j^*)\|^2 = 2f^\top Lf,$$

where $L = D - W$ is the graph Laplacian, and W is the k -NN affinity matrix. By Lemma 1, this equals the empirical Dirichlet energy of f on the k -NN graph. It is well known (Zhou et al., 2003; Belkin & Niyogi, 2008) that such a quadratic penalty is equivalent to Tikhonov regularization with respect to the graph Laplacian norm $\|f\|_L^2 = f^\top Lf$. In statistical learning theory, adding a Tikhonov penalty strictly reduces the variance of the estimator compared to the unregularized solution while keeping the bias term controlled. Thus MRIL enjoys smaller estimator variance than vanilla BC, which uses no such penalty.

(ii) Smoothness guarantee. Consider the continuum limit (Corollary 1): for i.i.d. samples $\{s_i^*\}$ from density p on a smooth manifold $\mathcal{M}$, the normalized penalty converges to

$$\int_{\mathcal{M}} \|\nabla f(s)\|^2 p(s)^2 d\text{vol}(s).$$

This is the weighted Dirichlet energy of f on $\mathcal{M}$. If this integral is finite, f belongs to the Sobolev space $H^1(\mathcal{M}, p^2)$, and in particular f is locally Lipschitz almost everywhere with

$$\|f(s^*) - f(s'^*)\| \leq C \|s^* - s'^*\| \quad \text{for } p\text{-a.e. neighbor pairs } s^*, s'^*.$$

Therefore minimizers of $\mathcal{L}_{\text{MRIL}}$ have uniformly bounded local Lipschitz constants along high-density regions of the state space. By contrast, minimizers of vanilla BC have no such constraint: any oscillatory interpolant that matches the training data exactly yields the same supervised risk, so arbitrarily large Lipschitz constants are possible.

(iii) Policy stability. Let $\Delta_t = \|s_t - s_t^*\|$ denote the deviation at time t . For Lipschitz dynamics T ,

$$\Delta_{t+1} \leq L_s \Delta_t + L_a \|f(s_t) - f^*(s_t^*)\|.$$

Decompose the action error:

$$\|f(s_t) - f^*(s_t^*)\| \leq \|f(s_t) - f^*(s_t)\| + \|f^*(s_t) - f^*(s_t^*)\|.$$

For vanilla BC, the first term $\|f(s_t) - f^*(s_t)\|$ is only minimized on the empirical distribution $P_{\mathcal{S}}$; off-distribution, it may be $O(1)$ regardless of Δ_t . The second term satisfies $\|f^*(s_t) - f^*(s_t^*)\| = O(\Delta_t)$ by smoothness of f^* . Hence the recursion can take the form

$$\Delta_{t+1} \leq L_s \Delta_t + L_a (O(1) + O(\Delta_t)),$$

which accumulates linearly in t .

⁴ L_s and L_a are the Lipschitz constants of the environment transition dynamics with respect to state and action, respectively.

For MRIL, the Laplacian penalty enforces

$$\|f(s^*) - f(s'^*)\| \leq C\|s^* - s'^*\| \quad \text{for neighbor pairs } (s^*, s'^*),$$

as shown in part (ii). Thus $\|f(s_t) - f^*(s_t)\| = O(h^2)$ (where h is the radius of the kernel bandwidth around the point s_t) by local-linear regression error bounds, and $\|f^*(s_t) - f^*(s_t^*)\| = O(\Delta_t)$ by smoothness of f^* . Combining these,

$$\Delta_{t+1} \leq L_s \Delta_t + L_a(O(h^2) + O(\Delta_t)).$$

Since the constant multiplying Δ_t is strictly smaller under the smoothness constraint, the cumulative error grows strictly slower than in vanilla BC. In particular, error growth is sublinear in the rollout horizon when h is small, whereas it is linear for vanilla BC.

Conclusion. Claims (i)–(iii) establish that MRIL yields lower variance, uniform smoothness control, and sublinear rollout error accumulation compared to vanilla BC, completing the proof. $\square$

Kernel choice. For the IC smoothness regularizer, we adopt a Gaussian kernel

$$w_i(s^*) \propto \exp\left(-\frac{\|s^* - s_i^*\|^2}{2h^2}\right), \quad \sum_{i \in \mathcal{N}_k(s^*)} w_i(s^*) = 1,$$

with bandwidth h set to the median distance to the k -th nearest neighbor across the dataset. This choice is standard in manifold regularization (Belkin & Niyogi, 2008; Zhou et al., 2003) and ensures that the graph Laplacian penalty converges to the Dirichlet energy in the continuum limit. In practice, we found this default to be stable across tasks, though other kernels (e.g., uniform k -NN or exponential decay) yield qualitatively similar results.

A.1.2 PROOF OF THEOREM 2

We begin with the required Lemmas establishing the spectral form of explicit Laplacian regularization and neighbor aggregation.

Lemma 2 (Spectral form of explicit Laplacian regularization). *Let L be the symmetric normalized graph Laplacian with eigenpairs $\{(\mu_j, u_j)\}_{j=1}^n$, where $0 = \mu_1 \leq \mu_2 \leq \dots \leq \mu_n \leq 2$. The minimizer of the penalized objective*

$$\mathcal{L}_\lambda(f) = \|f - a^*\|^2 + \lambda f^\top L f$$

has the closed-form expansion

$$f_\lambda = \sum_{j=1}^n \frac{1}{1 + \lambda \mu_j} \langle a^*, u_j \rangle u_j.$$

Thus λ directly determines the spectral filter $\phi_\lambda(\mu) = (1 + \lambda \mu)^{-1}$ applied to each Laplacian mode.

Proof. Diagonalize $L = U\Lambda U^\top$ with $U = [u_1, \dots, u_n]$ orthogonal and $\Lambda = \text{diag}(\mu_1, \dots, \mu_n)$. Write $f = Uc$, $a^* = Ub$ in this basis. The objective becomes

$$\|Uc - Ub\|^2 + \lambda c^\top \Lambda c = \sum_{j=1}^n (c_j - b_j)^2 + \lambda \mu_j c_j^2.$$

Minimizing each term yields $c_j = \frac{1}{1 + \lambda \mu_j} b_j$. Transforming back gives the stated expansion. $\square$

Lemma 3 (Spectral form of neighbor aggregation). *Let $S = D^{-1}A$ be the random-walk matrix of the k -NN graph, with adjacency A and degree D . For any prediction vector f , the neighbor-averaged prediction is $\hat{f} = Sf$. In the Laplacian eigenbasis, this corresponds to the spectral filter*

$$\hat{f} = \sum_{j=1}^n (1 - \mu_j) \langle f, u_j \rangle u_j,$$

i.e. $\phi_{\text{DARP}}(\mu) = 1 - \mu$.

Proof. By definition, $L = I - D^{-1/2}AD^{-1/2}$ and $S = D^{-1}A = I - L_{\text{rw}}$ where $L_{\text{rw}} = D^{-1}L$ is the random-walk Laplacian. Since L_{rw} and L share the same spectrum up to similarity transform, the eigenbasis $\{u_j\}$ diagonalizes S . Thus for each mode u_j , $Su_j = (1 - \mu_j)u_j$, yielding the claimed spectral filter. $\square$

Theorem 2 (iMRIL is parameter-free Laplacian regularization for BC (MRIL)). *Consider the symmetric normalized k -NN graph Laplacian L (defined in Section 2.2), with eigenpairs $\{(\mu_j, u_j)\}_{j=1}^n$, where $0 = \mu_1 \leq \mu_2 \leq \dots \leq \mu_n \leq 2$.*

The minimizers of the explicit MRIL objective (Section 2.2) and the implicit iMRIL objective (Section 2.3) have the following closed form expansions

$$f_{\text{MRIL}} = \sum_{j=1}^n \frac{1}{1 + \lambda \mu_j} \langle a^*, u_j \rangle u_j \quad \hat{f}_{\text{iMRIL}} = \sum_{j=1}^n (1 - \mu_j) \langle f, u_j \rangle u_j$$

iMRIL's neighbor aggregation step applies the fixed spectral filter $\phi_{\text{iMRIL}}(\mu) = 1 - \mu$ to the graph Laplacian L , preserving low-frequency modes and suppressing high-frequency modes. The congruence between $\hat{f}_{\text{iMRIL}}$ and f_{MRIL} shows that iMRIL is equivalent to a built-in form of Laplacian smoothing (MRIL) with effective $\lambda \approx 1$ in normalized units. Unlike explicit regularization, this implicit filter requires no additional hyperparameter tuning.

Proof. From Lemma 3, the neighbor aggregation operator $S = D^{-1}A$ acts on Laplacian eigenmodes u_j as

$$Su_j = (1 - \mu_j)u_j,$$

where μ_j are the normalized Laplacian eigenvalues. Thus in the graph Fourier basis, neighbor aggregation corresponds to multiplying each mode by the fixed spectral filter $\phi_{\text{DARP}}(\mu) = 1 - \mu$.

On the other hand, Lemma 2 shows that explicit Laplacian regularization with parameter λ yields the spectral filter $\phi_\lambda(\mu) = (1 + \lambda \mu)^{-1}$. Both filters downweight high-frequency modes ($\mu \gg 0$) while preserving low-frequency modes ($\mu \approx 0$). The key difference is that $\phi_\lambda(\mu)$ requires tuning λ , whereas $\phi_{\text{DARP}}(\mu)$ is parameter-free.

To see the equivalence, note that for small μ ,

$$\phi_{\text{DARP}}(\mu) = 1 - \mu \approx (1 + \mu)^{-1} = \phi_{\lambda=1}(\mu) \quad \text{up to } O(\mu^2) \text{ terms.}$$

Thus DARP can be interpreted as performing Laplacian smoothing with an effective regularization weight of order $\lambda \approx 1$ in normalized units. Moreover, for large μ , $\phi_{\text{DARP}}(\mu)$ damps high-frequency modes even more strongly by driving them toward zero, providing a sharper low-pass effect than explicit regularization.

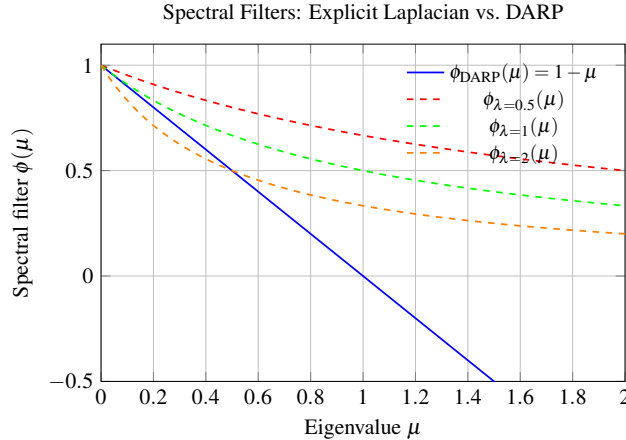


Figure 8: DARP achieves sharper low-pass filtering

Therefore, DARP's aggregation step is mathematically equivalent to implicit Laplacian regularization with fixed spectral filter ϕ_{DARP} , eliminating the need to tune λ explicitly. $\square$

DARP can therefore be viewed as a form of *locally adaptive implicit regularization*: rather than introducing an explicit global weight λ , its neighbor aggregation step enforces smoothness automatically through the graph structure. The effective regularization strength varies with local degree and neighborhood geometry, adapting to the density of the expert demonstrations. Spectrally, this corresponds to the fixed filter $\phi_{\text{DARP}}(\mu) = 1 - \mu$, which suppresses high-frequency modes more aggressively than any fixed explicit λ . Figure 8 illustrates this comparison, showing how DARP achieves sharper low-pass filtering without the need for hyperparameter tuning.

A.2 ADDITIONAL EXPERIMENTAL DETAILS

A.2.1 RETRIEVAL

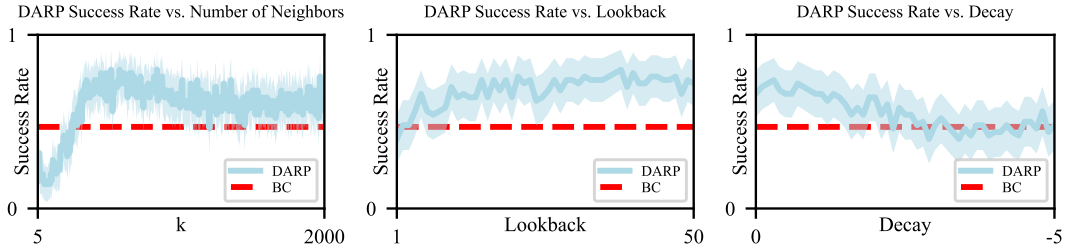


Figure 9: DARP performance analysis as retrieval hyperparameters are swept: (left) observe that the performance of a DARP model is poor when using few neighbors, reaches a global optimum when retrieving about 500 neighbors, and plateaus just above BC's success rate as k goes to the size of the dataset; (center) observe that the performance of a DARP model generally slightly improves as more history is considered, and only performs worse than BC when very little or no history is considered; (right) observe that the performance of a DARP model is sensitive to how much weight is applied to older observations when performing retrieval. Intuitively, if this decay is too high, DARP performance is nearly identical to having little to no lookback, performing worse than BC. Success rate is measured out of 50 trials on the Robosuite Stack environment. 95% confidence intervals are included.

The selection of the distance function $d(s_q, s_i^*)$ to select k neighbors is crucial. While we find that simple Euclidean distance between states can work, in our experiments, we use a slightly modified algorithm that takes advantage of the fact that we are working with sequences of states and incorporates history in our distance calculation.

Suppose we have a query trajectory $S_q = (\dots, s_{q,-1}, s_{q,0})$ where $s_{q,0}$ is the current query state s_q . Now suppose we want to calculate $d(s_q, s_i^*)$, where s_i^* is some state from the expert dataset. We first find the trajectory this state is from—call this S_j^* —and the index of s_i^* in this trajectory—call this t . Thus, s_i^* can be rewritten as $s_{j,t}^*$. Given some lookback parameter ℓ which denotes how many past states we want to consider, we get:

$$d(s_q, s_i^*) = \sum_{n=0}^{\ell-1} \|s_{q,-n} - s_{j,t-n}^*\|$$

This is simply the accumulation of Euclidean distances of the current and last $\ell - 1$ states from both the query trajectory and the source trajectory, assuming valid indices. Of course, in practice, we generally want to put more emphasis on more recent states, as we want them to be more influential in the selection of neighbors. Thus, given some rate of exponential decay $\gamma \geq 0$, we have

$$d(s_q, s_i^*) = \sum_{n=0}^{\ell-1} \|s_{q,-n} - s_{j,t-n}^*\| \cdot e^{-\gamma n}$$

See Figure 9 for an experimental analysis on how the success rate in an environment changes as these parameters are swept.

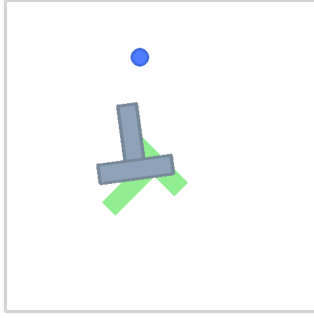


Figure 10: **Push-T Environment.** The goal is to control the blue circle to push the T-shaped block.

Method	Score
BC	48 ± 8
DARP	70 ± 8

Table 4: **Push-T Results.** Averaged over 100 trials, DARP outperforms BC.

A.2.2 CAN DARP HANDLE TASKS REQUIRING THE REPRESENTATION OF MULTI-MODAL ACTION DISTRIBUTIONS?

We test DARP’s ability to handle complex action distributions by evaluating on the Push-T task, as described in (Chi et al., 2024), which requires representing multi-modal action distributions (see Figure 10 for a visualization). For this experiment, DARP employs a Set Transformer head that predicts parameters of a Gaussian Mixture Model. We note that DARP with a GMM head is able to handle multi-modal distributions effectively, showing a 22% improvement over BC on the Push-T task (Q2) (see Table 4). This demonstrates that DARP can be further adapted to multi-modal action distribution modeling requirements.

A.2.3 CAN DARP HANDLE DISCONTINUOUS ENVIRONMENTS WHERE NEARBY STATES MAY REQUIRE OPPOSING ACTIONS?

A key concern for neighbor-based approaches is performance in environments with strong discontinuities, where states that are close in Euclidean distance may require drastically different actions. To address this concern, we design a stress test using a modified version of D4RL’s Umaze environment (see Figure 11 for a visualization).

Even in this deliberately challenging discontinuous environment, DARP achieves a 57% success rate compared to BC’s 25%. (Q3) (see Table 5) This suggests that the distance vectors and permutation-invariant aggregation help the model distinguish between appropriate and inappropriate neighbors, even when spatial proximity doesn’t guarantee action similarity.

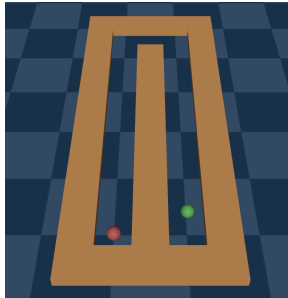


Figure 11: **Long maze environment.** The goal is to move a force-actuated ball from the green start to the red destination.

Method	Succ. (%)
BC	25
DARP	57

Table 5: **Long maze results.** Averaged over 100 trials, DARP significantly outperforms BC.

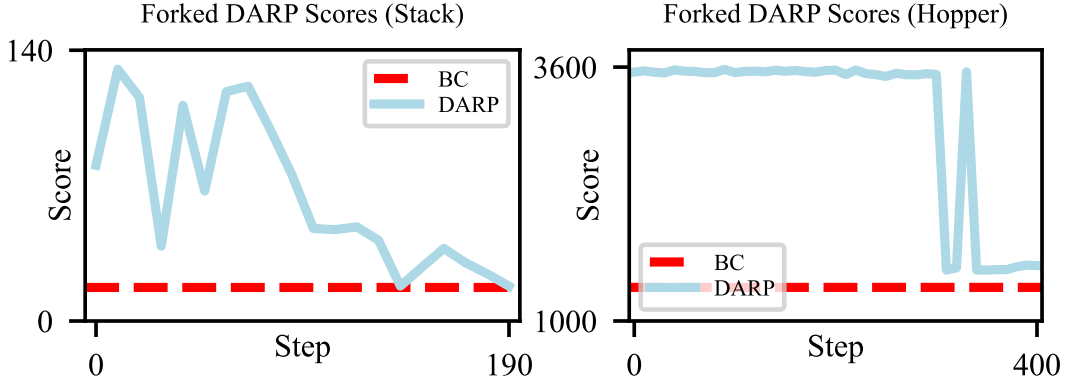


Figure 12: In two different tasks (the Robosuite Stack task and the MuJoCo Hopper task), we rollout a BC agent and create a fork of the environment every k steps (in this case, $k = 10$). Observe that, even as BC nears the end of its failing rollout, DARP is able to scale highly, and is only prevented from doing so about halfway through the Stack rollout and about 80% through the Hopper rollout.

A.2.4 CAN DARP RECOVER FROM BC ERROR?

In order to analyze DARP’s robustness to accumulated error, we roll out a BC agent in an environment in which we know it will fail, but every k steps, we create a fork of the environment and begin rolling out a DARP agent in that clone of the environment. The results (seen in Fig. 12) show that, even as BC approaches failure and drifts away from the support of expert demonstrations, DARP is able to recover and score very highly. This suggests that DARP indeed has superior robustness to accumulation of error and can perform well in the slightly out-of-distribution states that a failing BC agent drifts into.

A.2.5 COMPARISON WITH CCIL

Environment	Method	Improvement over BC (%)
Hopper	CCIL	32.6%
	DARP	53.2%
Walker	CCIL	84.1%
	DARP	84.1%
Ant	CCIL	12.6%
	DARP	84.5%
HalfCheetah	CCIL	5.4%
	DARP	418.7%

Table 6: **Relative improvement over BC compared to CCIL.** Comparing DARP against CCIL on standard MuJoCo benchmarks. DARP achieves higher or equal relative gains across all tasks.

As shown in Table 6, we evaluate our method against CCIL, a baseline that explicitly induces smoothness. We use the reported scores in the CCIL paper, and compare percent improvement over BC. Observe that DARP outperforms CCIL significantly on three out of four environments, with a particularly large margin on HalfCheetah (418.7% vs 5.4% improvement).

A.2.6 DISTANCE METRIC SENSITIVITY

While all experiments performed in this paper use Euclidean distance to choose nearby neighbors, it is natural to consider alternative metrics, like cosine similarity, especially in high-dimensional embeddings such as R3M. We find (as shown in Table 7) that DARP performs similarly – only 5% worse – when using cosine similarity rather than Euclidean distance. This suggests DARP is robust to the choice of distance metric used.

Method	Success Rate (%)
BC	47
DARP (Cosine Similarity)	70
DARP (Euclidean Distance)	75

Table 7: **Distance Metric Sensitivity.** Comparison of success rates using R3M features. DARP with Euclidean distance and cosine similarity perform similarly, beating BC by 28% and 23% respectively. Results are on the Robosuite Stack task.

A.2.7 DARP IN COMBINATION WITH DIFFUSION POLICY

Method	Stack	Threading	Peg Insertion
BC (MLP)	34	12	28
DARP (MLP)	62	38	40
Diffusion	52	44	54
DARP w/ Diffusion	82	72	66

Table 8: **DARP in combination with diffusion policy.** DARP provides significant improvements when applied to both standard MLP policies and Diffusion policies. Results are on the Robosuite Stack, Threading, and Peg Insertion tasks as success rate across 50 trials. States are state-based. Note that the number of demonstrations used is much less than that used in Table 2, so baseline BC and DARP numbers do not match.

While all reported experiments are performed with an MLP backbone, diffusion policy (Chi et al., 2024) has proven to be a state-of-the-art model class for imitation learning, particularly for manipulation tasks. Table 8 reveals that DARP is not mutually exclusive with diffusion and can be combined for an even more performant model. Using the DARP architecture with a diffusion backbone outperforms DARP with an MLP backbone by 20%, 34%, and 26% success rate increase for all three tasks respectively, beating standard BC by a total 48%, 60%, and 38% for the three tasks.

Method	Training (s/epoch)	Inference (s/step)
Diffusion	5.610	0.15700
DARP (MLP)	0.559	0.00437

Table 9: **Computational efficiency of DARP with an MLP backbone and diffusion policy.** Comparison of runtime costs between DARP (MLP) and diffusion policy. Diffusion policy is significantly more expensive, being $\approx 10\times$ slower in training and $\approx 36\times$ slower during inference.

Additionally, we empirically find that DARP with an MLP backbone is much faster than standard diffusion, particularly in inference – see Table 9.

A.2.8 DARP TRAINED ON HUMAN DEMONSTRATIONS

Method	Success Rate (%)
BC	45
DARP	60

Table 10: **Results with human demonstrations.** Comparison of success rates when training on human data rather than data collected by RL policies. Results are on the Robosuite Stack task.

The expert demonstrations used to train models for the MuJoCo and Robosuite environments are collected by an optimal Reinforcement Learning policy. It is crucial to ensure DARP maintains a performance gain in comparison to BC when trained on expert demonstrations collected by humans.

Indeed, when trained on human demonstrations on the Robosuite Stack task, DARP outperforms standard BC by 15% (see Table 10).

A.2.9 CHOICE OF RETRIEVAL HYPERPARAMETERS

Environment	Method	Score (Mean)
Hopper	BC	507.49
	DARP	805.69
Stack	BC	0.12
	DARP	0.31

Table 11: **Performance comparison when validation loss is used to select training epochs and retrieval hyperparameters.** Mean scores on Hopper and Stack environments. Observe that DARP maintains a performance gain in comparison to BC.

Figure 9 reveals that there are selections of retrieval parameters (for example, a very low number of neighbors) which cause DARP to perform worse than standard BC. However, we find that choosing retrieval hyperparameters that minimize validation loss is an effective strategy to find performant settings, see Table 11.

A.3 PSEUDOCODE

We provide pseudocode of the DARP algorithm, see Algorithm 1.

A.4 RUNTIME ANALYSIS

Environment	k	Train (s/epoch)	Test (s/step)
Hopper (1,000 datapoints)	BC	0.102	0.00127
	100	0.126	0.00413
	200	0.128	0.00418
	300	0.130	0.00420
	400	0.131	0.00421
	500	0.130	0.00419
Stack (4,200 datapoints)	BC	0.431	0.00139
	100	0.556	0.00437
	250	0.539	0.00442
	500	0.559	0.00437
	750	0.576	0.00433
	1000	0.591	0.00478
	1500	0.682	0.00452
	2000	0.758	0.00451

Table 12: **Runtime comparison across environments.** Training and testing speeds (in seconds) for Behavior Cloning (BC) and varying values of k on Hopper and Stack datasets.

As shown in Table 12, we analyze the computational overhead of DARP at both training-time and inference-time. Our analysis indicates that computation cost scales sub-linearly as k increases, and that inference time is tractable for real-time robotic application. For example, on the Robosuite Stack task at $k = 500$, inference takes approximately 0.00437 seconds per step on our hardware, corresponding to a control frequency higher than 230 Hz.

1242
1243
1244
1245
1246
1247
1248
1249
1250
1251
1252
1253
1254
1255
1256
1257
1258
1259
1260
1261
1262
1263
1264
1265
1266
1267
1268
1269
1270
1271
1272
1273
1274
1275
1276
1277
1278
1279
1280
1281
1282
1283
1284
1285
1286
1287
1288
1289
1290
1291
1292
1293
1294
1295

Algorithm 1 Difference-Aware Retrieval Policies

```

1: Input: Expert demonstrations  $\mathcal{D}^* = \{(s_j^*, a_j^*)\}$ , number of neighbors  $k$ 
2: Initialize:  $f$  parameters  $\theta$ 
3: if  $g$  is parametric then
4:   Initialize:  $g$  parameters  $\psi$ 
5: end if
6: // Training Loop
7: while not converged do
8:   Sample batch of query data  $(s_q^*, a_q^*) \sim \mathcal{D}^*$ 
9:   for each query pair  $(s_q^*, a_q^*)$  in batch do
10:    // Find  $k$ -Nearest Neighbors from the entire dataset  $\mathcal{D}^*$ 
11:     $\mathcal{N}_k(s_q^*) \leftarrow \text{argmin-}k_j d(s_q^*, s_j^*)$ 
12:    // Compute Neighbor-based Predictions
13:    for each neighbor index  $i \in \mathcal{N}_k(s_q^*)$  do
14:       $a'_i \leftarrow f_\theta(s_i^*, a_i^*, s_i^* - s_q^*)$ 
15:    end for
16:    // Aggregate Predictions
17:    if  $g$  is parametric then
18:       $\hat{a}_q \leftarrow g_\psi(\{a'_i\}_{i \in \mathcal{N}_k(s_q^*)})$ 
19:    else
20:       $\hat{a}_q \leftarrow g(\{a'_i\}_{i \in \mathcal{N}_k(s_q^*)})$ 
21:    end if
22:  end for
23:  // Update Parameters based on the batch loss
24:   $\mathcal{L} \leftarrow \sum_{(s_q^*, a_q^*) \in \text{batch}} \|\hat{a}_q - a_q^*\|^2$ 
25:  // Gradient descent step
26:   $\theta \leftarrow \theta - \alpha \nabla_\theta \mathcal{L}$ 
27:  if  $g$  is parametric then
28:     $\psi \leftarrow \psi - \alpha \nabla_\psi \mathcal{L}$ 
29:  end if
30: end while
31: Output: Trained parameters  $\theta$  and, if applicable,  $\psi$ 

```
